

СОДЕРЖАНИЕ

ЧАСТЬ 1. ОБЩАЯ ХАРАКТЕРИСТИКА БАЗЫ ПРАКТИКИ.....	3
1.1 Организация производства	3
1.2 Структура и функции подразделения базы практики	3
ЧАСТЬ 2. РЕАЛИЗАЦИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ.....	5
2.1 Сравнительный анализ аналогов проектируемой системы/подсистемы/модуля	5
2.2 Выбор технологии, среды и языка программирования для реализации ..	5
2.3 Анализ процесса обработки информации, выбор структур данных для ее хранения.....	6
2.4 Проектирование структур данных (ER-диаграмма базы данных, физическая модель БД).....	7
2.5 Проектирование интерфейса пользователя, выбор технологий и средств его реализации.....	7
2.6 Реализация выбранных методик, форм и паттернов проектирования и программирования для требуемого ПО.....	8
ЧАСТЬ 3. МЕРОПРИЯТИЯ ПО ОХРАНЕ ТРУДА И ТЕХНИКЕ БЕЗОПАСНОСТИ	10
3.1 Общие требования безопасности	10
3.2 Требования безопасности перед началом работы	10
3.3 Требования безопасности во время работы	10
3.4 Требования безопасности в аварийных ситуациях	11
ИСПОЛЬЗУЕМЫЕ ИСТОЧНИКИ ИНФОРМАЦИИ, ДОКУМЕНТЫ	12

ЧАСТЬ 1. ОБЩАЯ ХАРАКТЕРИСТИКА БАЗЫ ПРАКТИКИ

В качестве базы производственной практики выступает ООО «Малленом Системс». Компания была создана в 2011 году на базе команды ученых и программистов Санкт-Петербургского политехнического университета Петра Великого. Сегодня в штате числится более 100 сотрудников. В основе разрабатываемых систем лежат собственные решения на базе нейронных сетей и алгоритмы компании Cognex.

1.1 Организация производства

Информационные системы играют ключевую роль в деятельности предприятия, обеспечивая автоматизацию, управление данными и поддержку принятия решений. Разработка программного обеспечения строится на современном технологическом стеке:

- Платформы и языки: Используется платформа .NET 9.0.5 (C#) и среда разработки ASP.NET.
- Работа с данными: Применяются СУБД PostgreSQL и MS SQL Server. Взаимодействие с базами данных реализуется через технологию EntityFramework. Общение с сервисами происходит по REST API или SignalR.
- Развертывание и контроль: Для обеспечения работы сервисов применяются облачные решения и системы контейнеризации (Docker, Kubernetes). Управление исходным кодом ведется с использованием систем контроля версий, таких как Git, GitHub и GitLab.

1.2 Структура и функции подразделения базы практики

ИТ-инфраструктура является фундаментом для деятельности компании. В организации используются серверные мощности для вычислений и хранения данных, а также локальные и внешние сети.

Функции ИТ-подразделения включают:

- Администрирование инфраструктуры: Использование специализированного ПО для администрирования сетей и оборудования, такого как Ansible, Zabbix и Nagios.
- Управление ресурсами: Работа с системами управления контентом (CMS) для поддержки статического и динамического содержимого.
- Мониторинг: Настройка систем логирования для отслеживания работы приложений с использованием инструмента Serilog.
- Нормативный контроль: Соблюдение политик безопасности, регламентов проведения резервного копирования и правил работы с конфиденциальными данными.

ЧАСТЬ 2. РЕАЛИЗАЦИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

2.1 Сравнительный анализ аналогов проектируемой системы/подсистемы/модуля

В рамках выполнения индивидуального задания был проведен анализ существующих программных решений для мониторинга и генерации сетевого трафика. Основными критериями оценки выступали: функциональность, доступность, удобство интерфейса и возможность кастомной генерации пакетов. В качестве аналогов были рассмотрены:

1. Wireshark – стандарт в области графического анализа трафика с мощнейшим функционалом декодирования. Недостатки: избыточность для локальных задач, высокое потребление ресурсов памяти и отсутствие встроенного удобного генератора кастомных пакетов.
2. Tcpdump – легковесная консольная утилита. Недостатки: отсутствие графического интерфейса (GUI) и сложность визуального анализа больших потоков данных.
3. Библиотека Scapy – инструмент на языке Python, позволяющий захватывать и конструировать пакеты на низком уровне. Недостаток: отсутствие готовой графической оболочки для конечного пользователя.

Разрабатываемый программный комплекс занимает нишу между сложными профессиональными анализаторами и гибкими консольными утилитами, предоставляя мощь библиотеки Scapy в удобной визуальной оболочке.

2.2 Выбор технологии, среды и языка программирования для реализации

Учитывая специфику задачи (низкоуровневая работа с сетью), был выбран следующий технологический стек:

- Язык программирования: Python (версия 3.10+). Выбор обусловлен лаконичностью синтаксиса и наличием специализированных библиотек для работы с сетью.
- Сетевой движок: Библиотека Scapy (версия 2.5.0). Позволяет напрямую работать с «сырыми» сокетами (Raw Sockets) и послойно разбирать/формировать заголовки пакетов.
- Взаимодействие с ОС: Библиотека psutil. Используется для динамической привязки перехватываемого трафика к активным процессам Windows.
- Среда разработки: JetBrains PyCharm, предоставляющая удобные инструменты для отладки многопоточных приложений.
- Системная зависимость: Сетевой драйвер Npcap для перехвата пакетов в обход стандартного стека ОС Windows.

2.3 Анализ процесса обработки информации, выбор структур данных для ее хранения

Процесс обработки информации в разработанном ПО базируется на многопоточной архитектуре и разделен на этапы захвата, деинкапсуляции и фильтрации. Информация поступает с сетевого адаптера в виде байтовой последовательности. В модуле парсинга (PacketParser) происходит её разбиение на логические слои (Dissection). Алгоритм последовательно анализирует заголовки канального (Ethernet), сетевого (IP) и транспортного (TCP/UDP) уровней, извлекая необходимые поля (адреса, порты, флаги, TTL).

Для хранения информации в процессе работы выбрана структура «очередь» (Queue/List), выступающая буфером между потоком перехвата и потоком отрисовки интерфейса. Это гарантирует, что ни один пакет не будет потерян при высокой интенсивности трафика.

2.4 Проектирование структур данных (ER-диаграмма базы данных, физическая модель БД)

В связи со спецификой программного средства (анализатор трафика реального времени), использование классических реляционных баз данных (SQL) и построение ER-диаграмм не является целесообразным. Роль «физической модели данных» выполняют структуры в оперативной памяти и форматы экспорта:

1. Оперативная структура пакета (Словарь Python): Деинкапсулированные данные преобразуются в структуру Dictionary со следующими ключами: proto (протокол), src и dst (IP-адреса), src_port и dst_port (порты), flags (TCP флаги), ttl и length. Данная структура позволяет выполнять быстрый поиск ($O(1)$ – $O(n)$) и фильтрацию для отображения в таблице.
2. Структура агрегированной статистики: Для модуля аналитики используется древовидный словарь: {"TCP": int, "UDP": int, "ICMP": int, "Total": int, "ips": {ip: count}, "flags": {flag: count}}. Эта структура динамически обновляется и служит источником данных для отрисовки графиков.
3. Долговременное хранение: * Формат .PCAP – бинарный формат физического хранения сырых пакетов, обеспечивающий совместимость с промышленными стандартами (Wireshark).
- Формат .CSV – табличная структура данных (Время, Источник, Назначение, Протокол, Длина) для последующего статистического анализа.

2.5 Проектирование интерфейса пользователя, выбор технологий и средств его реализации

Для реализации пользовательского интерфейса (GUI) был выбран фреймворк PyQt6. Он обеспечивает кроссплатформенность, высокую скорость отрисовки компонентов и поддержку современных стилей (QSS). Концепция

интерфейса построена на принципах «Dashboard» (единое рабочее пространство) для минимизации переключений контекста.

Архитектура GUI разделена на 4 основные вкладки:

- Анализатор (SnifferTab): Содержит панель управления захватом, поле прикладной фильтрации (поиска) и таблицу QTableWidgetItem для вывода лога пакетов.
- Аналитика (AnalyticsTab): Интегрирована с библиотекой Matplotlib для визуализации 4 графиков реального времени (распределение протоколов, сетевая активность PPS, гистограмма IP и анализ TCP флагов).
- Генератор (GeneratorTab): Форма для конструирования пакетов (поля назначения, источника/spoofing, выбор протокола и флагов).
- Терминал (TerminalTab): Интерактивная консоль для выполнения Python-скриптов с библиотекой Scapy "на лету". Дизайн выполнен в темных тонах для снижения зрительной нагрузки.

2.6 Реализация выбранных методик, форм и паттернов проектирования и программирования для требуемого ПО.

Программная архитектура базируется на принципе модульности и разделения ответственности (Separation of Concerns). Применены следующие паттерны и методики:

1. Многопоточность (Multithreading): Реализована через класс QThread. Сетевые операции (перехват в SnifferThread, генерация в GeneratorThread) жестко изолированы от основного потока GUI. Это предотвращает блокировку интерфейса (GUI Freeze) при пиковых нагрузках трафика.
2. Паттерн Наблюдатель (Observer / Signals & Slots): Взаимодействие между сетевым ядром и интерфейсом осуществляется асинхронно через объекты pyqtSignal. Поток захвата генерирует событие, которое перехватывается графической оболочкой для безопасного обновления таблицы.

3. Локализация (i18n): Внедрен паттерн загрузки языковых конфигураций из JSON-файлов (ru.json, en.json), что обеспечивает гибкую смену языка интерфейса без перекомпиляции исходного кода.

ЧАСТЬ 3. МЕРОПРИЯТИЯ ПО ОХРАНЕ ТРУДА И ТЕХНИКЕ БЕЗОПАСНОСТИ

3.1 Общие требования безопасности

- К самостоятельной работе допускаются лица не моложе 18 лет, прошедшие вводный инструктаж и инструктаж на рабочем месте.
- Главные опасные и вредные производственные факторы для ИТ-специалиста: электромагнитное излучение, зрительное напряжение, статическая рабочая поза.

3.2 Требования безопасности перед началом работы

- Визуальный осмотр рабочего места: проверка исправности розеток, вилок, кабелей питания ПК.
- Работа с информационными ресурсами в организации строго регламентирована федеральным законодательством и локальными актами. Перед началом работы необходимо убедиться в соблюдении внутренних политик безопасности, включая правила создания паролей.

3.3 Требования безопасности во время работы

- Для снижения зрительного напряжения и статической нагрузки необходимо делать перерывы по 10–15 минут каждый час работы за монитором.
- Во время работы необходимо соблюдать правила цифровой гигиены и инструкции по работе с конфиденциальными данными.

3.4 Требования безопасности в аварийных ситуациях

- В случае появления запаха гари, дыма или искрения от техники — немедленно отключить оборудование от электросети.
- Сообщить о неполадках руководителю. В случае программных сбоев опираться на техническую документацию и регламенты проведения резервного копирования для восстановления работоспособности систем.

ИСПОЛЬЗУЕМЫЕ ИСТОЧНИКИ ИНФОРМАЦИИ, ДОКУМЕНТЫ

1. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (ред. от 04.11.2022).
2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Введ. 2018-07-01. – М.: Стандартинформ, 2017.
3. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. – Введ. 1980-01-01. – М.: Стандартинформ, 2010.
4. Типовая инструкция по охране труда при работе на персональном компьютере ТОИ Р-45-084-01 (утв. приказом Минсвязи РФ от 02.07.2001 № 162).
5. Бэрри, П. Изучаем программирование на Python / П. Бэрри. – М.: Эксмо, 2017. – 624 с.
6. Лутц, М. Изучаем Python / М. Лутц. – 5-е изд. – СПб.: Диалектика, 2019. – 832 с.
7. Методические рекомендации по оформлению рефератов, курсовых и дипломных работ (проектов). – Череповец: БПОУ ВО «ЧЛМТ», 2022.
8. Олифер, В. Г. Компьютерные сети. Принципы, технологии, протоколы: Учебник для вузов / В. Г. Олифер, Н. А. Олифер. – 5-е изд. – СПб.: Питер, 2020. – 992 с.
9. Прохоренок, Н. А. Python 3 и PyQt 6. Разработка приложений / Н. А. Прохоренок, В. А. Дронов. – СПб.: БХВ-Петербург, 2023. – 832 с.
10. Официальный сайт ООО «Малленом Системс» [Электронный ресурс]. – Режим доступа: <https://mallenom.ru/>
11. Npcap: Windows Packet Capture Library & Driver [Электронный ресурс]. – Режим доступа: <https://npcap.com/>

12. PyQt6 Reference Guide. Riverbank Computing [Электронный ресурс]. – Режим доступа: <https://www.riverbankcomputing.com/software/pyqt/>
13. Scapy 2.5.0 documentation [Электронный ресурс]. – Режим доступа: <https://scapy.readthedocs.io/en/latest/>
14. Официальная документация Python 3 [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/>